KI-ENNA: (E)IN (N)EURONALES (N)ETZ ZUM (A)USPROBIEREN

TRAINIERE DEINE EIGENE KÜNSTLICHE INTELLIGENZ

KI – STARTER

UNTERSCHEIDEN

Bringe einer KI bei, Dinge zu unterscheiden: Beschreibe deinem neuronalen Netz **GEGENSTÄNDE** (AUFGABEN). Aus diesen Beispielen lernt **KI-ENNA** (APP) selbst, welche Merkmale die Dinge unterscheiden.

KI – ENTDECKER

SPRECHEN

Lass deine KI Wörter verstehen: Du gibst **KI-ENNA** (APP) Beispielsätze vor. Damit lernt dein neuronales Netz über einen einfachen **TRANSFORMER** (AUFGABEN), welche Wörter zusammen einen Sinn ergeben können.

KI – CHAMPION

EIGENER CHATBOT

Programmiere einen eigenen Chatbot: In einem optionalen Workshop entwickelst Du einen lokalen und datenschutzkonformen **KI-ENNA Chatbot**, der nach deinen Vorgaben auf einem Raspberry Pi Computer läuft.

KI – STARTER

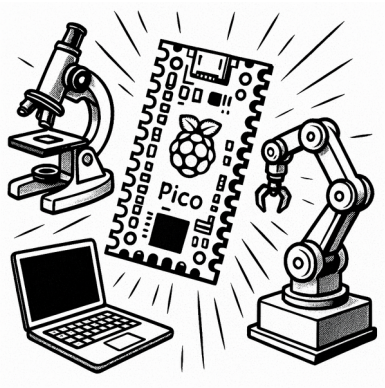
SEHEN

Bringe einer KI bei, Formen zu erkennen: Du trainierst dein neuronales Netz mit Kreisen, Dreiecken, Rechtecken und Quadraten. Mit einer Kamera kann **KI-ENNA** (APP) die **FORMEN** (AUFGABEN) selbst erkennen.

KI – ENTDECKER

KI-HARDWARE

Lass deine KI auf echter Hardware laufen: Du überträgst dein trainiertes neuronales Netz auf einen Raspberry Pi Pico Microcontroller. **KI-ENNA** (APP) steuert die Elektronik an und kann bspw. eine LED blinken lassen.



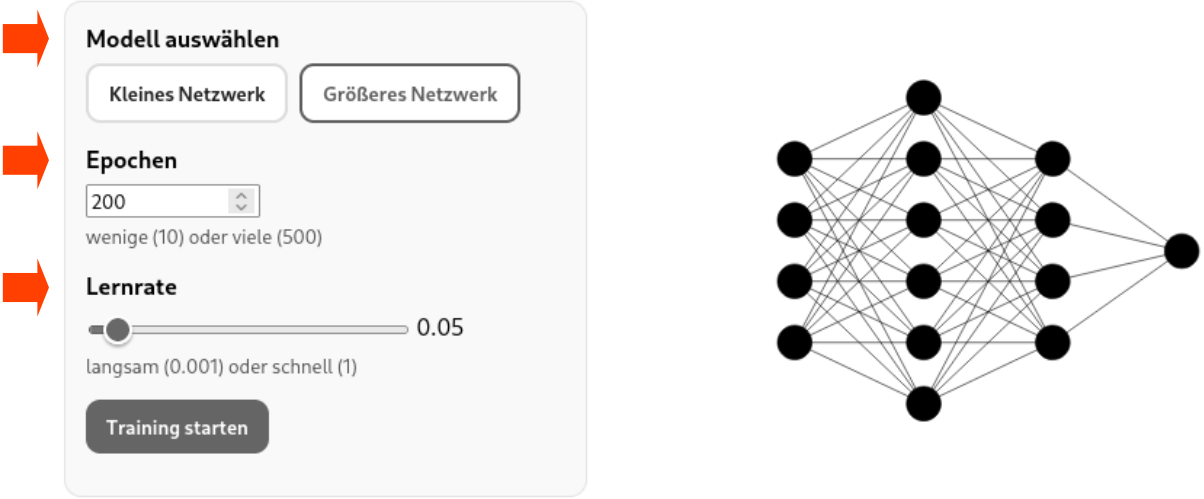
Weitere Informationen: **Homepage** (LINK), **Codes** (LINK), **Tutorial** (LINK) sowie **Open Source Referenzen** (LINK).

Prof. Dr. habil. Dennis Klinkhammer (2026)
www.STATISTICAL-THINKING.de

(1) KI-STARTER: Grundlagen neuronaler Netzwerke

Basiseinstellungen

In den Basiseinstellungen kann zwischen einem **kleinen neuronalen Netzwerk** und einem **größeren neuronalen Netzwerk** als Modell gewählt werden. Wenn dem neuronalen Netzwerk vier Eigenschaften (im Englischen: **Features**) vorgegeben werden, befinden sich vier sogenannte **Neuronen** links in der **Eingabeschicht**. Über die Anzahl an Neuronen in den nachfolgenden Schichten, den sogenannten verborgenen Schichten, werden diese Eigenschaften bis zur **Ausgabeschicht** verarbeitet. Das einzelne Neuron in der Ausgabeschicht trifft daraufhin eine Entscheidung, bspw. zwischen Nein (0) und Ja (1). Die Entscheidung eines jeden Neurons basiert auf **mathematischen Berechnungen**.



Epochen

Bei einem neuronalen Netzwerk bedeutet eine **Epoche**, dass das neuronale Netzwerk einmal **alle Trainingsbeispiele komplett durchlernt**. Danach schaut das neuronale Netzwerk, welche Aufgaben es gut gelöst hat und wo es noch Fehler gemacht hat, und versucht beim nächsten Durchgang besser zu werden. Mehrere Epochen helfen dem neuronalen Netzwerk also, **aus Fehlern zu lernen und immer genauer zu werden**.

Lernrate

Die **Lernrate** sagt bei einem neuronalen Netzwerk, **wie große Schritte beim Lernen gemacht** werden sollen. Wenn das neuronale Netzwerk Fehler entdeckt, kann es diesen **nur ein kleines bisschen korrigieren** oder **gleich sehr stark ändern**. Die Lernrate bestimmt also, ob das neuronale Netzwerk **langsam in kleinen Schritten** oder **schneller in größeren Schritten** aus seinen Fehlern lernt. Eine gute Lernrate hilft dem neuronalen Netzwerk, **ruhig und Schritt für Schritt** immer besser zu werden.

(2) KI-ENTDECKER: Grundlagen neuronaler Netzwerke mit Transformer

Vokabulargröße

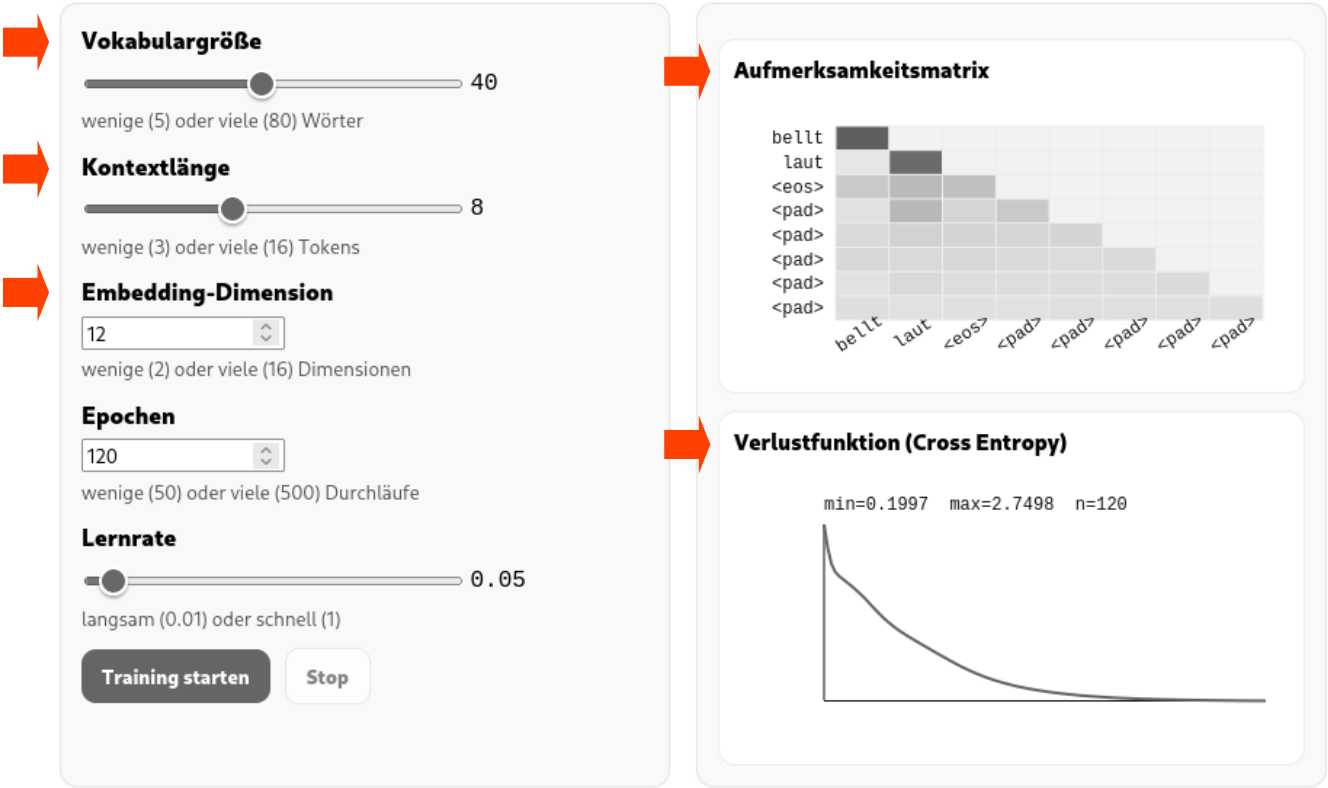
Die Vokabulargröße beschreibt, **wie viele verschiedene Wörter oder Wortteile ein Sprachmodell kennt**. Man kann sich das wie ein Wörterbuch vorstellen: Je größer das Wörterbuch ist, desto mehr Wörter kann man verstehen und benutzen.

Kontextlänge

Die Kontextlänge sagt, **wie viele Wörter sich das Modell gleichzeitig merken und beachten kann**, wenn es einen Text liest oder neuen Text schreibt. Wenn sich das neuronale Netzwerk mit Transformer viele Wörter aus dem Satz davor merken kann, versteht es den Text besser.

Embedding-Dimension

Die **Embedding-Dimensionen** beschreiben, **wie viele Zahlen** das Modell benutzt, um die **Eigenschaften eines Wortes** abzubilden. Der Transformer verarbeitet die Embeddings **kontextabhängig** und verbessert sie während des Trainings.



Aufmerksamkeitsmatrix

Eine **visualisierte Aufmerksamkeitsmatrix** zeigt, **auf welche Wörter ein Sprachmodell beim Lesen eines Satzes besonders achtet**, sodass man sehen kann, welche Wörter für das Verständnis anderer Wörter wichtig sind.

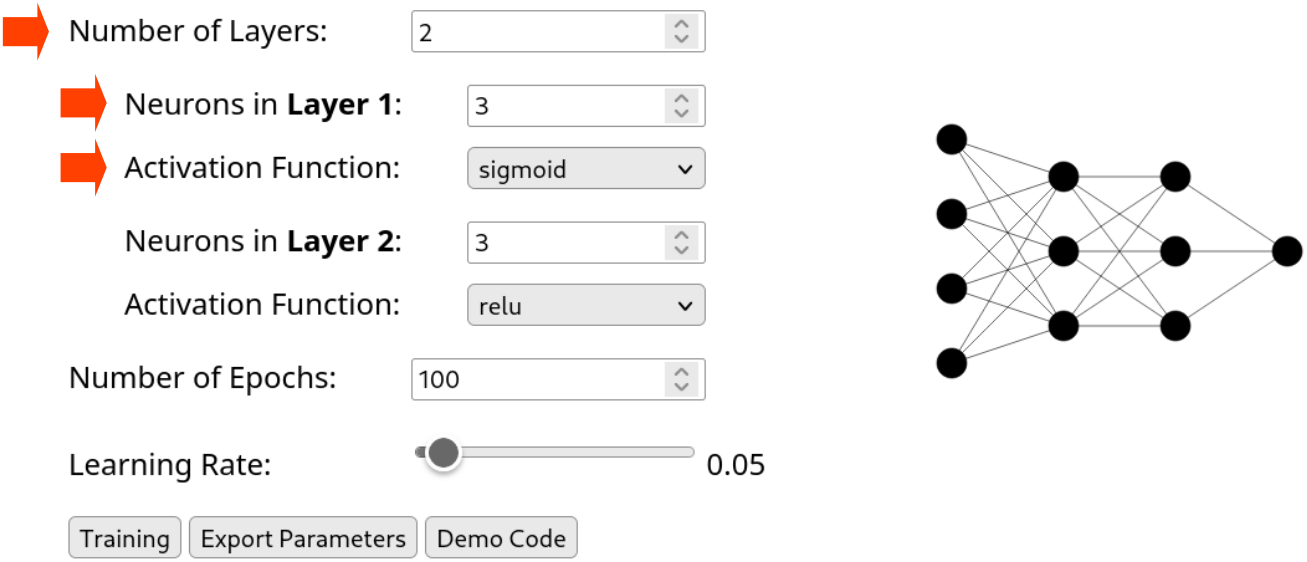
Verlustfunktion (Cross Entropy)

Die **Verlustfunktion** ist eine **Zahl**, die misst, **wie viele Fehler das Modell noch macht**, damit das neuronale Netzwerk mit Transformer weiß, wie stark es seine Einstellungen ändern muss, um beim nächsten Lernen besser zu werden.

(3) KI-ENTDECKER: Grundlagen neuronaler Netzwerke auf einem Microcontroller

Erweiterte Einstellungen

In einer erweiterten Benutzeroberfläche können die **Anzahl der Schichten und Neuronen** sowie die **zugrundeliegenden Aktivierungsfunktionen** als Grundlage der mathematischen Berechnungen flexibel ausgewählt werden.



Parameterexport

In einem neuronalen Netzwerk hat jede Verbindung ein **Weight (w)**, das festlegt, wie stark eine Eingabe das Ergebnis beeinflusst – wichtige Informationen bekommen dabei größere Gewichte. Der **Bias (b)** ist ein Wert zur Weitergabe des Ergebnisses an das nächste Neuron. Diese Parameter können gemeinsam mit der **Architektur des trainierten neuronalen Netzwerks** und den **Normalisierungsparametern**, welche die Eingabewerte auf eine ähnliche Größenordnung bringen, über den **Demo Code** und mit **Thonny** (LINK) auf einen **Raspberry Pi Pico** Microcontroller überführt werden.

```
# Weights and Biases
w1 = [[-1.5547062114777843,-1.3149971434877115,5.916686620164458,4.305657314788336],
      [-2.0180324474081726,-1.4754254071975048,5.487588010656587,3.093915529016862],
      [-0.4972859634586839,-0.37628226249263264,-0.29252167181959465,0.161481677576104]]
b1 = [-1.5425636000868739,-1.6150043602531652,0]
w2 = [[0.24305619453999483,0.2835407876615999,-0.2786254586652932],
      [2.5506628106670415,2.4127642846906636,-0.16029087303574518],
      [-0.6844416367022176,-0.625188964670778,-0.4491823173814271]]
b2 = [0.6445739527748703,-1.164931916810375,2.623466014502224]
w3 = [[-0.28908100746756343,2.649028627941983,-2.1384683508612454]]
b3 = [-2.8377447372111377]

# Neural Network Architecture
l_out = dense(3, x, w1, b1, 'sigmoid')
l_out = dense(3, l_out, w2, b2, 'relu')
l_out = dense(1, l_out, w3, b3, 'sigmoid')

# Normalization Parameters
X_MINS = [4.9,2,3,1]
X_MAXS = [7.9,3.8,6.9,2.5]
```